

AgentScope 框架说明文档

面向开发者的多智能体应用构建平台

1. 概览 (Overview)

AgentScope 是一个专为开发者设计的创新性多智能体（Multi-Agent）协作平台。其核心目标是为构建具有高可靠性和易用性的多智能体应用提供坚实的基础设施。AgentScope 通过标准化的消息机制、强大的模型封装和灵活的智能体编排，使得开发者能够轻松构建从简单的对话机器人到复杂的分布式多智能体系统。

核心优势

- 易用性：** Pythonic 的编程体验，开箱即用。
- 高可靠：** 内置容错重试与类型检查机制。
- 全功能：** 覆盖 RAG、记忆、工具调用、评估等全生命周期。

适用场景

- 多模态对话系统
- 复杂任务分解与执行 (ReAct)
- 分布式智能体集群模拟
- 自动化代码生成与调试

2. 安装 (Installation)

AgentScope 需要 Python 3.10 或更高版本。您可以选择从 PyPI 直接安装，或从源码编译以获取最新特性。

基础安装

Terminal

```
1
2 # 从 PyPI 安装稳定版
3 pip install agentscope
4
5 # 验证安装
6 python -c "import agentscope; print(agentscope.__version__)"
7
```

源码安装与扩展依赖

对于需要使用模型 API、工具函数或进行开发的场景，建议安装扩展依赖。

安装方式	命令	说明
源码安装	<code>pip install -e .</code>	克隆 GitHub 仓库后在根目录执行
全功能依赖	<code>pip install agentscope[full]</code>	包含所有模型 API 和工具库支持 (Win/Mac/Linux)
开发依赖	<code>pip install agentscope[dev]</code>	包含测试工具和文档生成工具

3. 核心概念 (Core Concepts)

AgentScope 的架构围绕三个核心对象构建：**消息 (Message)**、**模型 (Model)** 和 **智能体 (Agent)**。理解它们的关系是开发的关键。

3.1 消息与内容块 (Message & Content)

所有的智能体交互都通过 `Msg` 对象进行封装。AgentScope 引入了结构化的内容块（Content Block）机制，以支持多模态和复杂工具调用。

Msg 对象结构

一个标准的 `Msg` 对象包含以下关键字段：

- name** (str): 发送者名称。
- role** (str): 角色类型 ("user", "assistant", "system")。
- content** (str | list): 消息内容，可以是纯文本或内容块列表。
- metadata** (dict): 结构化数据或额外元数据。

内容块类型 (Content Blocks)

为了处理多模态数据，`content` 字段可以包含以下类型的 Block：

Block 类型	用途	数据结构示例
<code>TextBlock</code>	纯文本内容	<code>{type: 'text', text: '...'}</code>
<code>ImageBlock</code>	图像数据	支持 <code>Base64Source</code> 或 <code>URLSource</code>
<code>ToolUseBlock</code>	工具调用请求	

```
{type: 'tool_use', name: 'search',  
input: {...}}
```

ThinkingBlock

思维链过程

```
{type: 'thinking', thinking: '...'}
```

3.2 模型与格式化器 (Model & Formatter)

AgentScope 提供了统一的模型接口 `ChatModelBase`，屏蔽了不同后端 API (OpenAI, DashScope, Gemini, Ollama) 的差异。

支持的模型后端

- **OpenAI Compatible:** 支持 GPT-4, o1, DeepSeek 等符合 OpenAI 接口规范的模型。
- **DashScope (阿里云):** 支持 Qwen (通义千问) 系列，包括 Qwen-Max, Qwen-VL。
- **Gemini (Google):** 支持 Gemini-Pro, Flash 等，具备强大的长上下文处理能力。
- **Ollama:** 支持本地部署的开源模型 (Llama 3, Mistral)。
- **Anthropic:** 支持 Claude 3.5 Sonnet 等模型。

Prompt Formatter

不同模型对消息格式的要求不同（如 Multi-user 场景下的 System Prompt 位置）。

`agentscope.formatter` 模块提供了自动化的格式转换：

- **DashScopeMultiAgentFormatter:** 自动处理多智能体对话历史的拼接。
- **OpenAIMultiAgentFormatter:** 适配 OpenAI 格式的多角色交互。
- **TruncatedFormatter:** 自动根据 Token 上限截断历史消息。

3.3 智能体 (Agent)

智能体是交互的主体。所有智能体均继承自 `AgentBase`。核心方法包括：

- `reply(msg)`：接收消息并生成回复。
- `observe(msg)`：仅接收消息但不回复（用于群聊旁听）。

内置智能体类型

类名	描述	典型用途
UserAgent	人类用户代理	处理 CLI 或 Web UI 的用户输入
ReActAgent	推理与行动智能体	工具调用、复杂任务规划、RAG 检索

4. ReActAgent 与 Agent 的区别与选型指南

在 AgentScope 中，[AgentBase](#) 是所有智能体的抽象基类，定义了最基本的交互接口；而 [ReActAgent](#) 则是基于 ReAct (Reasoning + Acting) 范式的全功能实现。正确选择基类能显著提升开发效率。

4.1 类层级与核心差异 (Class Hierarchy & Differences)

AgentScope 的智能体体系分为三层：通用基类、ReAct 基类和 ReAct 实现类。它们在抽象程度和内置能力上存在显著差异。

特性维度	AgentBase	ReActAgent
设计定位	最轻量的通用接口，仅定义 <code>reply</code> 和 <code>observe</code> 。	"Batteries-included" 的旗舰实现，开箱即用。
核心循环	开发者完全自定义 (自由度 100%)。	内置 <code>Reasoning -> Acting -> Observing</code> 闭环。
Hook 支持	基础 hooks (Reply/Observe/Print)。	扩展 hooks (+Reasoning/Acting)，支持精细控制。
工具与记忆	需手动集成。	原生支持 <code>Toolkit</code> ， <code>Memory</code> ， <code>Plan</code> 。

4.2 ReActAgent 的扩展能力 (Capabilities)

`ReActAgent` 集成了构建复杂 Agent 所需的八大核心能力，使其能够胜任从简单问答到复杂任务规划的各种场景。

基础增强 (Core Enhancements)

- **Realtime Steering**: 支持实时转向, 可随时打断 reply 并触发 `handle_interrupt`。
- **Memory Compression**: 支持记忆压缩, 基于 `CompressionConfig` 实现自动摘要机制。
- **Parallel Tool Calls**: 支持并行工具调用, 通过 `parallel_tool_calls=True` 开启。
- **Structured Output**: 支持结构化输出, 通过 `structured_model` 指定 Pydantic Schema。

高级扩展 (Advanced Extensions)

- **Fine-grained MCP**: 支持细粒度 MCP 控制, 连接并管理各类 MCP 工具与服务 (`agentscope.mcp`)。
- **Meta Tool**: 支持智能体工具管理, 允许 Agent 动态调整工具库 (`Toolkit`)。
- **Long-Term Memory**: 支持自我控制的长期记忆, 主动进行知识检索与记录 (`LongTermMemory`)。
- **State Management**: 支持自动状态管理, 内置会话与状态维护机制 (`Session`)。

参考资料: [AgentScope Agent Tutorial](#)

4.3 选型建议 (Selection Guide)

在实际工程中, 建议遵循以下原则进行选型:

选择 AgentBase

当且仅当:

- 你需要实现**非 ReAct 范式**的逻辑 (如纯状态机、简单的消息转发器)。
- 你的 Agent **不需要 LLM** 参与 (如规则引擎 Agent)。
- 你需要极度的轻量化和对执行流的完全掌控。

选择 ReActAgent

绝大多数场景的首选:

- 构建基于 LLM 的**智能助手**或**Copilot**。
- 需要使用**工具 (Tools)**、**RAG** 或 **记忆 (Memory)**。
- 需要处理多步推理或复杂任务规划。

4.4 最小代码骨架 (Code Skeleton)

通过代码可以更直观地看到两者的复杂度差异:

Implementation Comparison

```
1
2 # 场景 A: 使用 AgentBase 自定义简单回声机器人
3 class EchoAgent(AgentBase):
```

```

4     def __init__(self, name: str):
5         super().__init__(name=name)
6
7     async def reply(self, x: Msg = None) -> Msg:
8         # 完全自定义的逻辑，无内置 LLM 调用
9         return Msg(self.name, content=f"Echo: {x.content}")
10
11     # 场景 B: 使用 ReActAgent 构建全功能 AI 助手
12     # 无需定义类，直接实例化配置即可
13     agent = ReActAgent(
14         name="Assistant",
15         model=model_config,      # 接入 LLM
16         toolkit=tools,          # 注入工具
17         memory=long_term_memory, # 挂载记忆
18         parallel_tool_calls=True, # 开启并发
19         plan_notebook=notebook  # 开启规划
20     )
21

```

5. 高级特性 (Advanced Features)

5.1 工具与 MCP (Tools & MCP)

AgentScope 深度集成了 **Model Context Protocol (MCP)**，允许智能体无缝连接外部数据和工具。

- **StdIOStatefulClient**: 通过标准输入输出连接本地 MCP Server（保持状态）。
- **HttpStatelessClient/HttpStatefulClient**: 通过 SSE 或 HTTP 连接远程 MCP 服务。
- **MCPToolFunction**: 将 MCP 工具自动转换为 AgentScope 可调用的函数对象。

5.2 记忆与长期记忆 (Memory)

智能体不仅拥有临时的对话上下文 (`InMemoryMemory`)，还支持持久化的长期记忆 (`LongTermMemoryBase`)。

Mem0 集成

通过 `Mem0LongTermMemory`，AgentScope 支持基于向量数据库的语义检索记忆。智能体可以主动调用 `record_to_memory` 和 `retrieve_from_memory` 工具来管理知识。

5.3 评估与基准测试 (Evaluation)

内置的评估模块 (`agentscope.evaluate`) 支持对智能体性能进行定量分析。

- **ACEBenchmark:** 内置 ACE 评测数据集支持。
- **RayEvaluator:** 基于 Ray 的分布式并行评测，加速大规模测试。
- **Metric:** 支持数值型 (`ACEAccuracy`) 和分类型指标。

5.4 Embedding 服务

支持文本、图像、视频的多模态 Embedding 生成，用于 RAG 或记忆检索。支持 DashScope, OpenAI, Gemini 等主流后端。

6. Workflow 体系 (Workflow System)

AgentScope 提供了灵活的工作流编排能力，支持从简单的线性对话到复杂的动态多智能体协作。通过组合不同的 `Agent` 和 `Pipeline`，开发者可以构建出适应各种业务场景的 Workflow。

模式 (Mode)	拓扑结构	通信机制	典型场景	关键差异
Conversation	线性/交互 (A↔B)	显式传递 / MsgHub	客服、单一任务执行	最基础单元，注重上下文连贯性
Debate	环形/迭代 (A↔B↔C)	MsgHub 广播	复杂决策、代码 Review、多视角修正	引入"裁判"角色，通过多轮迭代收敛结果
Concurrent	扇入/扇出 (Fan-out)	独立执行 (Async Gather)	批量数据处理、投票、独立子任务	无状态依赖，追求高吞吐量
Routing	分支选择 (Select)	结构化输出 / 工具调用	意图识别、专家系统分发	依赖 Router 的分类能力，需定义清晰的边界
Handoffs	层级嵌套 (Host-Worker)	动态创建 & 工具回调	复杂任务分解、人机协作 (HITL)	Orchestrator 动态管理 Worker 生命周期

6.1 典型 workflow 详解

1. Conversation & MsgHub

对话模式不仅限于双人交互。 `MsgHub` 上下文管理器实现了消息的**自动广播**，使得群聊模式下无需显式编写 point-to-point 的传递逻辑。

MsgHub 广播示例

```
1
2  async with MsgHub(participants=[alice, bob, charlie]) as hub:
```

```
3 # 任何在 hub 内产生的消息都会自动广播给其他 participants
4 await alice() # Alice 发言, Bob 和 Charlie 自动调用 observe()
5 await bob() # Bob 发言
6
```

2. Multi-Agent Debate

辩论模式引入了**多轮迭代**和**评判机制**。通常包含持不同观点的 Debaters 和一个总结性的 Moderator (Judge)。关键在于 Moderator 需要在每轮结束时评估是否达成共识。

易错点提示

- **无限循环**：必须设置最大回合数 (Max Round) 或明确的终止条件 (Termination Condition)。
- **上下文污染**：如果辩论轮次过多，需配合 `TruncatedFormatter` 防止 Context Window 溢出。

3. Routing (路由)

路由模式决定了"谁来处理下一个任务"。AgentScope 支持两种路由实现：

结构化输出路由

Router 输出 JSON/Pydantic 对象，包含 `next_step` 字段。

适用：确定性的状态流转。

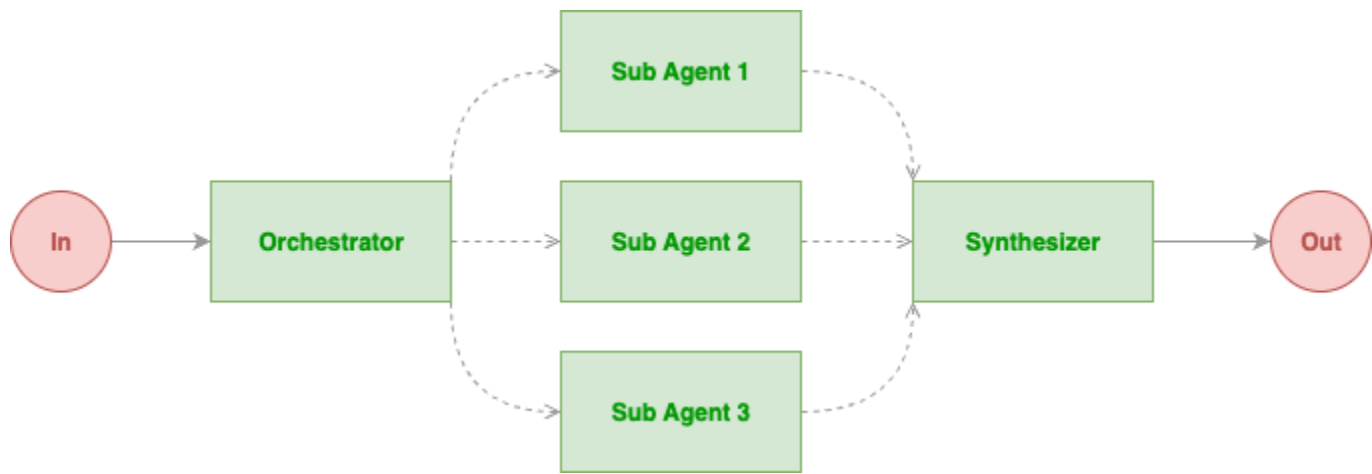
工具调用路由

将下游 Agent 封装为 Tool，Router 通过 `call_tool` 触发执行。

适用：不仅选择 Agent，还需传递特定参数。

4. Handoffs (Orchestrator-Workers)

这是处理复杂长任务的核心模式。Orchestrator 不直接干活，而是负责**分解任务**并**动态创建** Worker Agent 来执行。



Orchestrator-Workers 协作流程

Worker 完成任务后，其结果通过 `ToolResponse` 回传给 Orchestrator，后者整合结果并决定下一步行动。这种模式非常适合结合 **Sandbox** 执行不安全的操作。

7. 编排与规划 (Orchestration & Planning)

AgentScope 将"怎么做"(Execution) 与"做什么"(Strategy) 解耦为 **Pipeline** 和 **Plan** 两个模块。

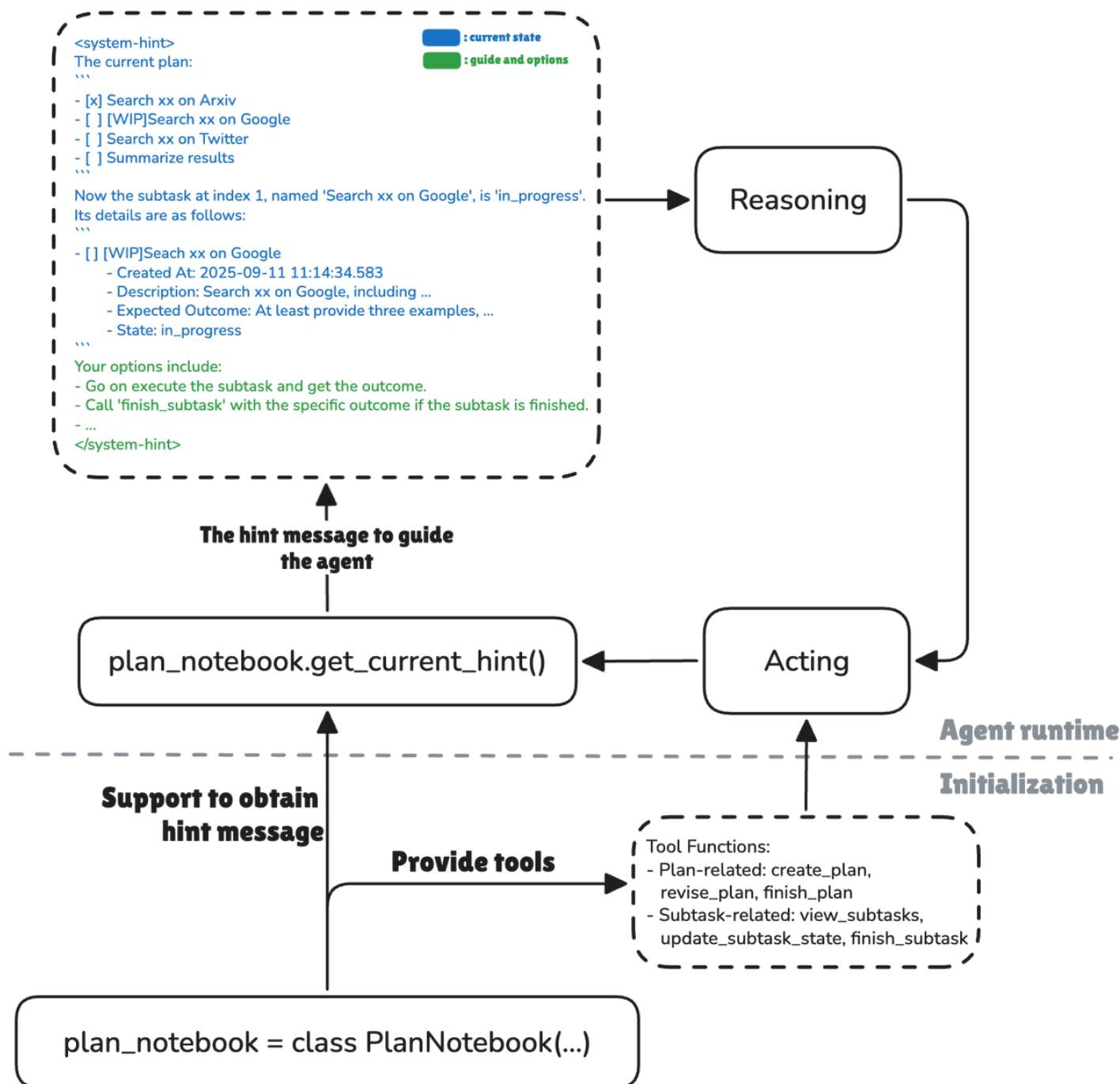
7.1 Pipeline：编排语法糖

Pipeline 是一组用于简化 Agent 调用的工具函数，本质上是**执行流的拓扑控制**。

Pipeline 类型	行为描述	代码映射
<code>sequential_pipeline</code>	串行执行，上一个输出即下一个输入	<code>res = await C(await B(await A(x)))</code>
<code>fanout_pipeline</code>	并行执行，同一输入分发给多个 Agent	<code>res = await asyncio.gather(A(x), B(x))</code>

7.2 Plan：任务计划本

`PlanNotebook` 赋予了 Agent **过程管理**的能力。与 Pipeline 的硬编码流程不同，Plan 允许 Agent 在运行时**动态调整**步骤。



ReAct Agent 与 PlanNotebook 的交互

- **System Hint:** PlanNotebook 会自动将当前进度（如 "Subtask 1: Done, Subtask 2: Todo"）注入到 Agent 的 System Prompt 中。
- **工具集:** 提供 `create_plan` , `update_subtask_state` , `revise_current_plan` 等工具，让 Agent 自主管理进度。

7.3 协同范式：Pipeline + Plan

最强大的模式是将两者结合：**Plan** 负责宏观的里程碑管理，**Pipeline** 负责微观的执行路径。

示例：复杂调研任务

1. **Plan 阶段**：Orchestrator 创建计划 ["搜索信息", "并行总结", "撰写报告"]。
2. **Pipeline 执行 - 步骤 1**：调用 `SearchAgent` (Routing)。
3. **Pipeline 执行 - 步骤 2**：使用 `fanout_pipeline` 启动多个 `SummaryAgent` 并行处理搜索结果 (Concurrent)。
4. **Plan 更新**：Orchestrator 标记步骤 2 完成，进入步骤 3。

8. 扩展机制 (Extensibility)

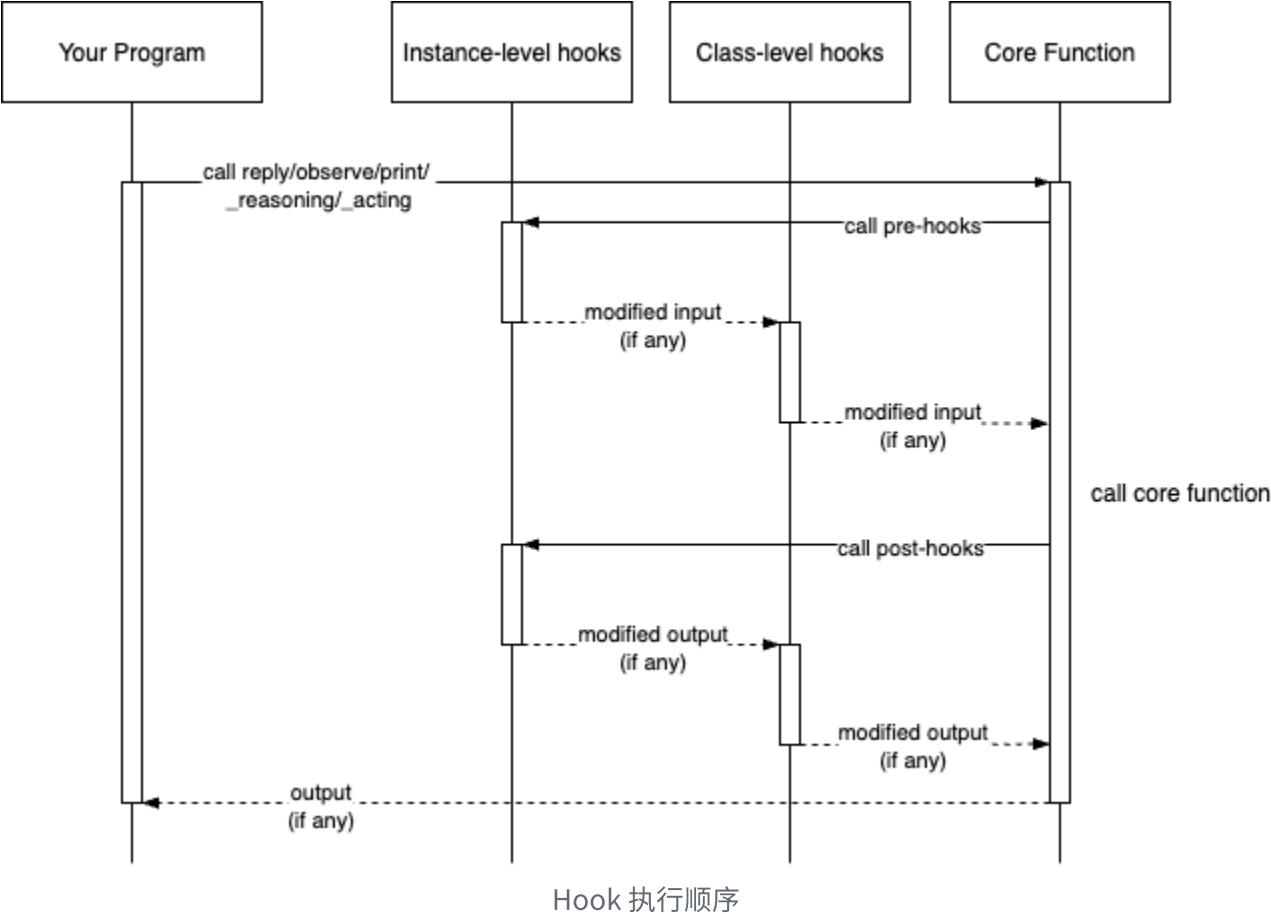
8.1 A2A Agent (Agent-to-Agent)

A2A 协议用于跨进程、跨网络的 Agent 通信。通过 `A2AAgent`，本地 Agent 可以像调用本地对象一样调用远程微服务化的 Agent。

- **Agent Card**：远程 Agent 的"身份证"，包含 URL、能力声明等。
- **Resolver**：获取 Agent Card 的策略。
 - `FileAgentCardResolver`：读取本地 JSON 配置文件。
 - `NacosAgentCardResolver`：从 Nacos 注册中心动态发现。
 - `WellKnownAgentCardResolver`：从标准 URL 路径获取。

8.2 Agent Hooks

Hooks 是介入 Agent 生命周期的切面编程 (AOP) 机制，常用于日志注入、鉴权、审计和可视化埋点。



Hook 类型	触发时机	典型用途
pre/post_reply	回复消息前后	修改输入 Prompt，过滤敏感输出，计费统计
pre/post_acting	工具调用前后 (ReAct)	阻断高危操作，替换工具参数

Hooks 支持**实例级** (Instance-level) 和 **类级** (Class-level) 注册。执行顺序遵循：Class Pre -> Instance Pre -> Function -> Instance Post -> Class Post。

9. 环境集成与沙箱 (Environment & Sandbox)

为了让 Agent 安全地执行代码、操作文件或浏览网页，AgentScope Runtime 提供了隔离的 Sandbox 环境。Sandbox 可以作为工具集 (Toolkit) 集成到 ReActAgent 中。

9.1 Sandbox 类型与部署

执行环境

- **Base Sandbox:** Python/Shell 执行。
- **Filesystem Sandbox:** 文件读写管理。
- **Browser Sandbox:** 网页浏览与交互。
- **Mobile Sandbox:** Android 模拟器操作。

部署模式

- **Local Docker:** 本地容器化隔离。
- **Sandbox Service:** 远程 HTTP 服务，支持 K8s 集群。
- **AgentBay:** 云端托管沙箱。

9.2 在 Workflow 中集成 Sandbox

在 Handoffs 模式中，最佳实践是将 Sandbox 封装在 Worker Agent 中。Worker 在沙箱内执行具体动作，保证了宿主环境的安全。

9.2.1 勘误：向沙箱添加 MCP 服务器的正确参数结构

在调用 `add_mcp_servers` 方法时，传递的配置字典必须包含顶级键 `mcpServers`，否则 Runtime 将无法正确解析服务配置（此处与官方文档保持一致）。下方代码展示了包含该键的正确写法，并补充了同时添加多个服务的扩展示例。

Worker 使用 Sandbox 工具 (修正版)

```
1
2  # 1. 连接沙箱服务
3  sandbox_service = SandboxService()
4  await sandbox_service.start()
5  sandboxes = sandbox_service.connect(session_id="task_1", sandbox_types=["base"])
6  base_box = sandboxes[0]
7
8  # 2. 将沙箱方法注册为 Agent 工具
9  toolkit = Toolkit()
10 # 自动发现并注册 run_shell_command, run_ipython_cell 等工具
11 for tool in base_box.list_tools():
12     toolkit.register_tool_function(tool)
13
14 # 3. 创建 Worker Agent
15 worker = ReActAgent(
16     name="Coder",
17     sys_prompt="You can write and execute code to solve problems.",
18     toolkit=toolkit, # 注入沙箱工具
19     model=model
20 )
21
22 # 4. MCP 集成 (修正后)
23 # 注意：必须通过 "mcpServers" 顶级键包裹服务配置
```

```
24  server_configs = {
25      "mcpServers": {
26          "time": {
27              "command": "uvx",
28              "args": ["mcp-server-time"]
29          },
30          # 扩展示例：添加第二个 MCP 服务 (如 Sandbox MCP)
31          "sandbox": {
32              "command": "uvx",
33              "args": [
34                  "--from", "agentscope-runtime", "runtime-sandbox-mcp",
35                  "--type=base",
36                  "--base_url=http://127.0.0.1:8000"
37              ]
38          }
39      }
40  }
41  base_box.add_mcp_servers(server_configs)
42
```

10. API 导航 (API Index)

以下是核心模块的 API 索引，供查阅详细参数：

模块	功能摘要
agentscope.agent	智能体基类、ReAct 智能体、Hook 机制
agentscope.message	消息对象、多模态内容块定义
agentscope.model	DashScope, OpenAI, Gemini 等模型封装
agentscope.memory	短期与长期记忆管理
agentscope.mcp	Model Context Protocol 客户端与工具
agentscope.evaluate	Agent 性能评估与基准测试